

Lab 2, Understanding OLS Through Simulation

Population, Sampling, and the Sampling Distribution of the OLS Estimator

2026-09-15

Contents

Introduction	1
Building the population	2
Mother's education	2
The error term and the true parameters	3
Putting the population together	3
A picture of the population	3
Drawing one sample and running OLS	4
Predicted values and residuals	5
The algebraic properties of OLS	6
The sampling distribution of the OLS estimator	7
Averaging across samples	8
Visualizing the sampling distribution	8
Summary and questions to think about	10

Introduction

In Lab 1 you learned how to work with objects, vectors, and data frames in R. This lab puts that foundation to a different use. Instead of importing a dataset written by someone else, we are going to build one ourselves, from scratch, so that we know the true relationship hiding inside it before we ever run a regression.

This matters because in real research you never see the true population relationship. You see one sample and you use OLS to estimate it. Here we get to play a trick that nature never allows an applied economist. We will create an entire population of one million mothers and babies, decide the true relationship between a mother's education and her baby's birthweight ourselves, and then pretend we are researchers who only get to see a sample of that population. Because we know the answer key in advance, we can check exactly how well OLS recovers it.

By the end of this lab you should be able to do the following.

1. Simulate a population from a known data generating process.
2. Draw a random sample and estimate a simple linear regression with `lm()`.
3. Compute and interpret predicted values and residuals.
4. Verify, numerically, the algebraic properties that any OLS regression line must satisfy.
5. Explain what a sampling distribution is by generating one yourself through repeated sampling.

```
library(ggplot2)
```

We also fix the random number seed so that everyone in the class, running this file on their own laptop, gets the same numbers. This is standard practice whenever randomness is involved in an analysis you want others to be able to reproduce.

```
set.seed(345)
```

Building the population

We are going to invent a population of one million mother and child pairs. The variable `mothereduc` is the mother's years of education and `birthweight` is the baby's weight in ounces. We are the ones deciding how these two variables relate to each other, through the following population regression function.

$$\text{birthweight}_i = \beta_0 + \beta_1 \text{mothereduc}_i + u_i$$

Here β_0 and β_1 are fixed numbers that describe the true population relationship, and u_i is the error term, the part of birthweight that has nothing to do with the mother's education. Genetics, nutrition, prenatal care, and plain chance all live inside u_i . We will draw u_i from a normal distribution centered at zero, which is the classical assumption you have seen in your econometrics lectures.

Mother's education

The lab asks for mother's education to come from a normal distribution with a mean of 12 years, restricted so that no one has less than 0 years of schooling or more than 23. A plain normal distribution has no natural floor or ceiling, so drawing straight from `rnorm()` would occasionally hand us a mother with negative years of education or with 40 years of education, neither of which makes sense. We fix this by drawing from a normal distribution and rejecting, then redrawing, any value that falls outside the 0 to 23 range. This gives us a genuinely normal shape within those bounds rather than a distribution with an artificial pile of observations stacked at the boundaries, which is what would happen if we simply capped extreme values instead of redrawing them.

We assume a standard deviation of 3 years for mother's education. This is not given directly in the assignment, so we are choosing a value that keeps almost the whole distribution comfortably inside the 0 to 23 window while still leaving some genuine spread across mothers.

```
draw_truncated_normal <- function(n, mean, sd, lower, upper) {  
  values <- numeric(0)  
  while (length(values) < n) {  
    remaining <- n - length(values)  
    candidates <- rnorm(remaining * 1.2, mean = mean, sd = sd)  
    candidates <- candidates[candidates >= lower & candidates <= upper]  
    values <- c(values, candidates)  
  }  
  values[1:n]  
}
```

The function draws a few more candidate values than we still need, keeps only the ones that land inside the allowed range, and repeats until it has collected enough. We draw a few extra each round purely so the loop runs only once or twice rather than many times, since most draws already fall inside the range.

```
N <- 1000000
```

```
mothereduc <- draw_truncated_normal(  
  n = N,  
  mean = 12,  
  sd = 3,
```

```

lower = 0,
upper = 23
)

summary(mothereduc)

```

```

##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## 0.002283  9.977072 12.001261 11.999902 14.025743 22.994351

```

The summary confirms the minimum and maximum sit inside the 0 to 23 boundary and the mean lands close to 12, as intended.

The error term and the true parameters

Now we pick the two numbers that make up the true, unknown-in-real-life relationship. We set $\beta_0 = 104$ and $\beta_1 = 1.5$, meaning a newborn whose mother has no schooling weighs 104 ounces on average, and each additional year of the mother's education adds 1.5 ounces to the baby's expected birthweight. We also draw the error term from a normal distribution with a mean of zero and a standard deviation of 20 ounces, which captures everything about birthweight that education does not explain.

```

beta0_true <- 104
beta1_true <- 1.5
sigma_error <- 20

error <- rnorm(N, mean = 0, sd = sigma_error)

```

Putting the population together

We now generate `birthweight` exactly according to the population regression function above, and store both variables in a data frame called `population`.

```

birthweight <- beta0_true + beta1_true * mothereduc + error

population <- data.frame(mothereduc, birthweight)

summary(population)

```

```

##      mothereduc      birthweight
## Min.   : 0.002283  Min.   : 22.56
## 1st Qu.: 9.977072  1st Qu.:108.20
## Median :12.001261  Median :122.05
## Mean   :11.999902  Mean   :122.03
## 3rd Qu.:14.025743  3rd Qu.:135.84
## Max.   :22.994351  Max.   :219.11

```

An average birthweight a little above 120 ounces, or roughly 7.6 pounds, matches what you would expect to see in real hospital data, which is a reassuring sign that our assumed parameters are reasonable.

A picture of the population

Plotting all one million points would be slow and would look like a solid block of ink, so we plot a random subset of 3000 pairs purely for visual clarity. The full population of one million observations is still what we sample from later.

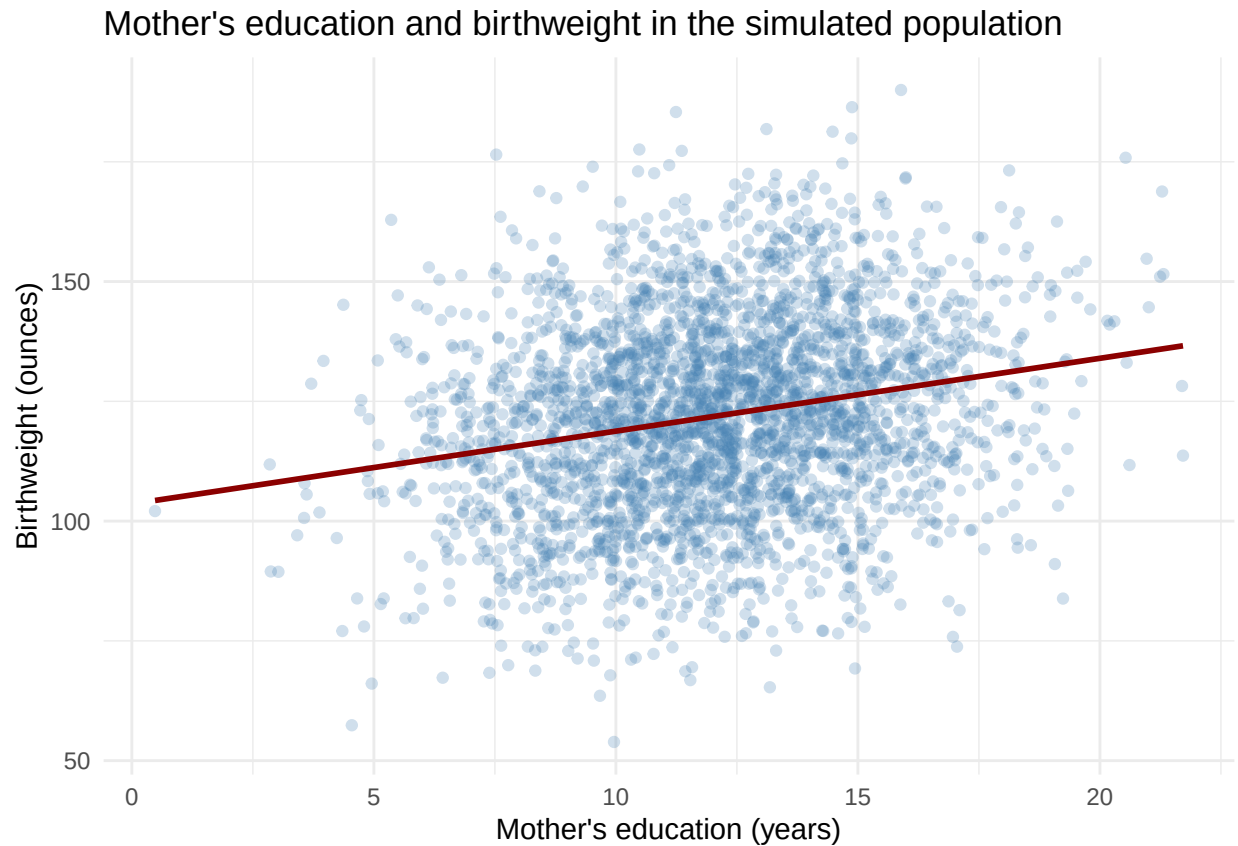
```

plot_sample <- population[sample(1:N, 3000), ]

ggplot(plot_sample, aes(x = mothereduc, y = birthweight)) +

```

```
geom_point(alpha = 0.25, color = "steelblue") +
geom_smooth(method = "lm", se = FALSE, color = "darkred") +
labs(
  title = "Mother's education and birthweight in the simulated population",
  x = "Mother's education (years)",
  y = "Birthweight (ounces)"
) +
theme_minimal()
```



The upward slope in the picture is the true β_1 at work, though it is loose rather than tight because most of the variation in any single baby's birthweight comes from the error term, not from the mother's education.

Drawing one sample and running OLS

A real researcher never gets to see the population we just built. They get one sample, usually a few thousand observations from a survey or a hospital records office, and they use OLS to estimate β_0 and β_1 from that sample alone. We now step into that researcher's shoes and draw a single random sample of 10,000 observations from our population of one million.

```
sample_data <- population[sample(1:N, 10000), ]

fit <- lm(birthweight ~ mothereduc, data = sample_data)
summary(fit)

##
## Call:
## lm(formula = birthweight ~ mothereduc, data = sample_data)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -89.191 -13.467   0.102  13.656  76.453
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  104.7841     0.8199  127.80  <2e-16 ***
## mothereduc    1.4615     0.0664   22.01  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 19.99 on 9998 degrees of freedom
## Multiple R-squared:  0.04621,    Adjusted R-squared:  0.04611
## F-statistic: 484.4 on 1 and 9998 DF,  p-value: < 2.2e-16

beta0_hat <- coef(fit)[1]
beta1_hat <- coef(fit)[2]

beta0_hat

## (Intercept)
##      104.7841

beta1_hat

## mothereduc
##      1.46147
```

The estimated intercept and slope are close to the true values of 104 and 1.5, though not identical to them. That gap is entirely expected. Every sample is a different random draw from the population, so every sample produces a slightly different estimate. The interpretation of $\hat{\beta}_1$ is that among mothers in this sample, one additional year of education is associated with about 1.46 more ounces of birthweight, holding nothing else fixed since this is a simple, one-regressor model.

Predicted values and residuals

Once we have $\hat{\beta}_0$ and $\hat{\beta}_1$, the fitted regression line lets us generate a predicted birthweight for every mother in the sample, whether or not that prediction matches what actually happened to her baby.

$$\widehat{birthweight}_i = \hat{\beta}_0 + \hat{\beta}_1 mothereduc_i$$

```
sample_data$birthweight_hat <- beta0_hat + beta1_hat * sample_data$mothereduc

sample_data$birthweight_hat_check <- fitted(fit)

all.equal(sample_data$birthweight_hat, sample_data$birthweight_hat_check)

## [1] TRUE
```

We computed the predicted values two ways, once by hand using the formula above and once with R's built in `fitted()` function, and they agree exactly, which is a useful sanity check whenever you compute something R can also compute for you.

The residual is simply the gap between what actually happened and what the line predicted.

```
sample_data$residual <- sample_data$birthweight - sample_data$birthweight_hat
head(sample_data[, c("mothereduc", "birthweight", "birthweight_hat", "residual")], 10)
```

```
##      mothereduc birthweight birthweight_hat residual
## 456492    9.748045   115.86618      119.0306  -3.164381
## 900060   15.935912   154.56664      128.0739   26.492707
## 428363    8.564938   141.26111      117.3015   23.959633
## 905894   13.852870   140.24686      125.0296   15.217225
## 447763   14.755159   131.23935      126.3483    4.891051
## 846718   10.080732   146.67131      119.5168   27.154537
## 127094   15.169518   128.40460      126.9539    1.450726
## 660945    8.775881   125.90646      117.6098    8.296690
## 38354     4.867212    88.38087      111.8974  -23.516491
## 416973   16.056832   125.41227      128.2507   -2.838388
```

A positive residual means that particular baby was heavier than the line predicted given the mother's education, and a negative residual means the baby was lighter than predicted.

The algebraic properties of OLS

OLS is not an arbitrary line drawn by eye. It is the specific line that minimizes the sum of squared residuals, and that minimization problem forces a handful of algebraic properties to hold exactly, for any data set, every single time. You have already seen these properties derived on paper in lecture. Here we check that our own regression obeys every one of them.

```
x <- sample_data$mothereduc
y <- sample_data$birthweight
yhat <- sample_data$birthweight_hat
uhat <- sample_data$residual

xbar <- mean(x)
ybar <- mean(y)
```

Property 1. The residuals sum to zero.

```
sum(uhat)
```

```
## [1] 2.355847e-09
```

Property 2. The regressor and the residual are uncorrelated, so their cross product sums to zero.

This is the orthogonality condition that OLS is built to satisfy in the first place, since minimizing the sum of squared residuals with respect to β_1 leads directly to this condition.

```
sum(x * uhat)
```

```
## [1] 2.837004e-08
```

Property 3. The regression line passes exactly through the point of means.

```
predicted_at_means <- beta0_hat + beta1_hat * xbar
c(predicted_at_means = predicted_at_means, ybar = ybar)
```

```
## predicted_at_means.(Intercept)          ybar
##                122.2851                122.2851
```

Property 4. The average of the predicted values equals the average of the actual values.

This follows directly from Property 1, since the residuals are simply $y_i - \hat{y}_i$ and their sum is zero.

```
c(mean_yhat = mean(yhat), mean_y = ybar)
```

```
## mean_yhat    mean_y
## 122.2851    122.2851
```

Property 5. The predicted values and the residuals are uncorrelated.

Since $\hat{y}_i$ is just a linear function of x_i , this property is really a repackaging of Properties 1 and 2 rather than a separate fact about the data.

```
sum(yhat * uhat)
```

```
## [1] 2.8746e-07
```

Property 6. Total variation splits cleanly into explained and unexplained parts.

The total sum of squares equals the explained sum of squares plus the residual sum of squares, and R^2 is simply the explained share of that total.

```
TSS <- sum((y - ybar)^2)
ESS <- sum((yhat - ybar)^2)
RSS <- sum(uhat^2)
```

```
c(TSS = TSS, ESS_plus_RSS = ESS + RSS)
```

```
##          TSS ESS_plus_RSS
## 4186918    4186918
```

```
r_squared_manual <- ESS / TSS
```

```
c(r_squared_manual = r_squared_manual, r_squared_lm = summary(fit)$r.squared)
```

```
## r_squared_manual    r_squared_lm
## 0.04620941         0.04620941
```

None of the sums above land on a number that is typed exactly as zero. You will instead see something like $1e-11$ or smaller. That is not a mistake in the algebra, it is floating point arithmetic. A computer stores numbers with a finite number of digits, and after ten thousand multiplications and additions a residue this tiny is unavoidable. For every practical purpose a number that small is zero, and the two sides of Property 6 above should match to every digit that matters.

The sampling distribution of the OLS estimator

The single sample above gave us one pair of estimates, $\hat{\beta}_0$ and $\hat{\beta}_1$. A natural question follows. If we had happened to draw a different sample of 10,000 mothers, would we have gotten the same estimates? Almost certainly not exactly, since sampling is random. The question that actually matters for econometric theory is whether OLS is right on average across all the samples we could have drawn, and how spread out the estimates tend to be from one sample to the next. That average and that spread together describe what is called the sampling distribution of the estimator.

We cannot observe this in a real research project because we only ever get one sample. Here, because we control the population, we can simply repeat the sampling exercise thousands of times and watch the sampling distribution emerge with our own eyes.

We draw 10,000 independent samples, each of size 10,000, run a regression on every single one, and store the resulting $\hat{\beta}_0$ and $\hat{\beta}_1$. To keep the loop fast we use `.lm.fit()`, a bare bones version of `lm()` that skips the extra output `lm()` normally prepares and simply returns the coefficients, which is all we need here. This chunk is cached so that once you have run it, knitting the document again will not repeat all ten thousand regressions unless the code actually changes.

```

set.seed(345)

n_sims <- 10000
n_sample <- 10000

beta0_hats <- numeric(n_sims)
beta1_hats <- numeric(n_sims)

for (i in 1:n_sims) {
  idx <- sample.int(N, n_sample)
  x_i <- mothereduc[idx]
  y_i <- birthweight[idx]
  design_matrix <- cbind(1, x_i)

  fit_i <- .lm.fit(design_matrix, y_i)

  beta0_hats[i] <- fit_i$coefficients[1]
  beta1_hats[i] <- fit_i$coefficients[2]
}

```

Averaging across samples

```

results_table <- data.frame(
  parameter = c("beta0", "beta1"),
  true_value = c(beta0_true, beta1_true),
  monte_carlo_average = c(mean(beta0_hats), mean(beta1_hats)),
  monte_carlo_sd = c(sd(beta0_hats), sd(beta1_hats))
)

knitr::kable(results_table, digits = 4)

```

parameter	true_value	monte_carlo_average	monte_carlo_sd
beta0	104.0	103.9786	0.8249
beta1	1.5	1.5045	0.0666

The average of ten thousand separate OLS estimates lands almost exactly on the true parameter value we used to build the population. That is what it means for OLS to be unbiased. It does not promise that any single sample gives you the right answer. It promises that the estimator is not systematically too high or too low, so that errors in one direction from one sample tend to cancel errors in the other direction from another sample.

Notice also that the Monte Carlo standard deviation of the `beta1_hats` is close to the standard error that a single `lm()` summary reported back in the one sample section. That is not a coincidence. The standard error reported by `lm()` is itself an estimate of exactly this sampling spread, produced from a single sample using statistical theory rather than from watching ten thousand samples directly.

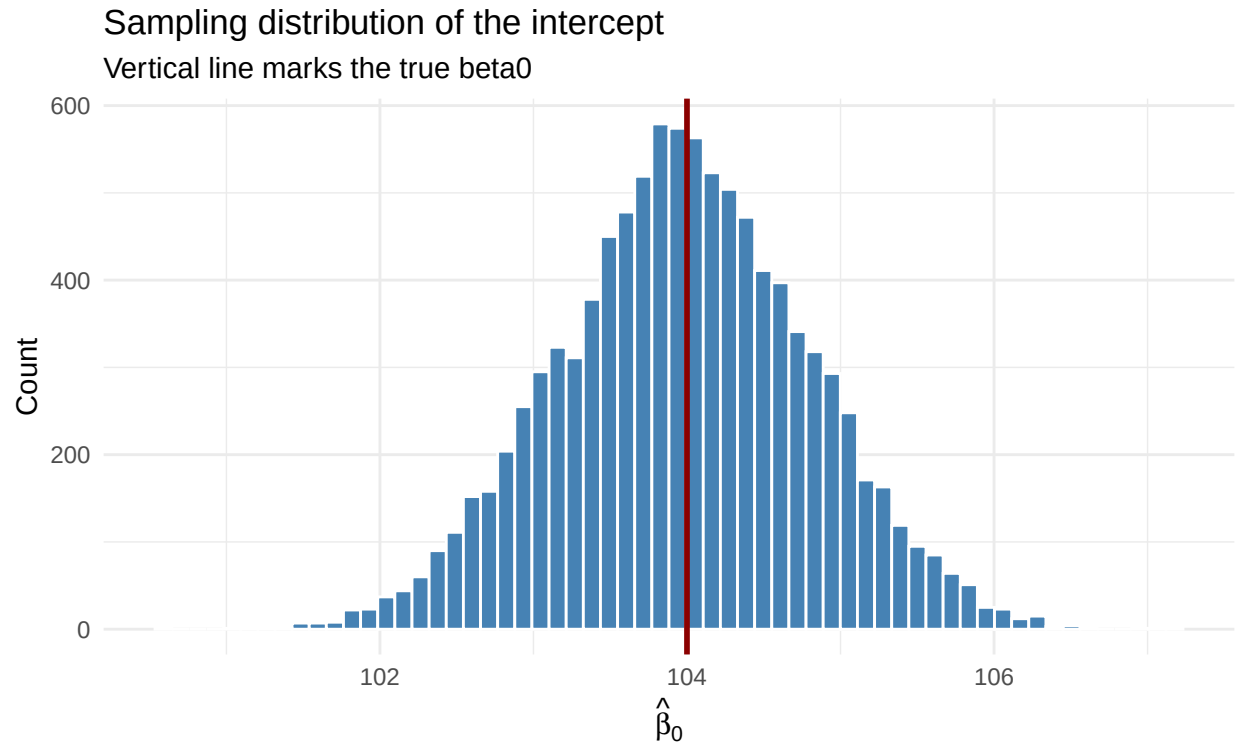
Visualizing the sampling distribution

```

ggplot(data.frame(beta0_hats), aes(x = beta0_hats)) +
  geom_histogram(bins = 60, fill = "steelblue", color = "white") +
  geom_vline(xintercept = beta0_true, color = "darkred", linewidth = 1) +

```

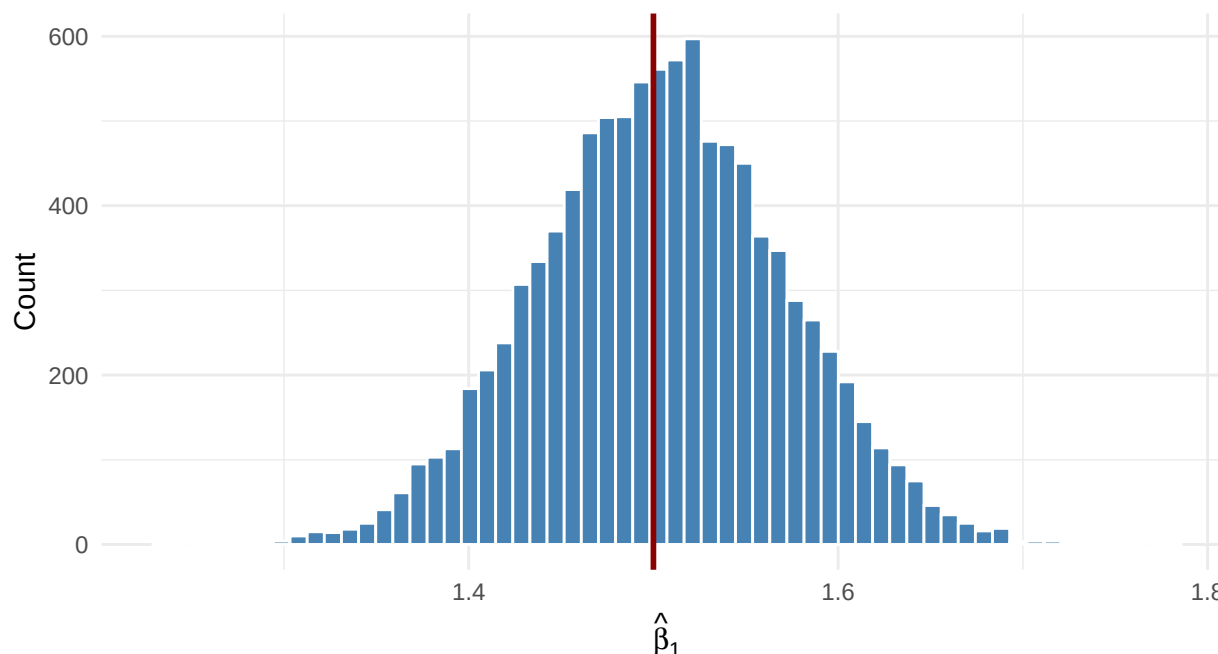
```
labs(
  title = "Sampling distribution of the intercept",
  subtitle = "Vertical line marks the true beta0",
  x = expression(hat(beta)[0]),
  y = "Count"
) +
theme_minimal()
```



```
ggplot(data.frame(beta1_hats), aes(x = beta1_hats)) +
  geom_histogram(bins = 60, fill = "steelblue", color = "white") +
  geom_vline(xintercept = beta1_true, color = "darkred", linewidth = 1) +
  labs(
    title = "Sampling distribution of the slope",
    subtitle = "Vertical line marks the true beta1",
    x = expression(hat(beta)[1]),
    y = "Count"
  ) +
  theme_minimal()
```

Sampling distribution of the slope

Vertical line marks the true β_1



Both histograms are centered almost exactly on the true parameter value and both look approximately bell shaped, which is the normal sampling distribution that econometric theory predicts for the OLS estimator under these classical assumptions.

Summary and questions to think about

We built a population of one million mothers and babies with a birthweight equation we wrote ourselves, drew a single sample and estimated it with OLS, confirmed by hand that the estimator satisfies every algebraic property it is supposed to, and then repeated the sampling exercise ten thousand times to see the sampling distribution of the estimator directly rather than just taking the theory on faith.

Before you submit this lab, work through the following questions in the space below the code.

1. Change `sigma_error` to 40 and rerun the Monte Carlo section. What happens to the spread of the `beta1_hats` histogram, and why does that make sense given what the error term represents.
2. Change `n_sample` inside the Monte Carlo loop to 500 instead of 10,000, keeping everything else fixed. What happens to the spread of the sampling distribution this time, and how does that connect to what you have learned about larger samples producing more precise estimates.
3. In the single sample section, `beta1_hat` was not exactly 1.5 even though 1.5 is the true value built into the population. Explain in your own words why that gap does not mean OLS failed.
4. Property 2 states that the regressor and the residual are uncorrelated in the sample OLS actually estimated. Does this mean the error term u_i in the true population model is also guaranteed to be uncorrelated with $mothereduc_i$. Explain the difference between these two statements.